

ICS 214 Lab 1

Welcome to the Lab for ICS214!

How the labs for this course work:

Labs will assume that you are tracking along with the modules. Modules are set up to explain the concepts and the language features used in each lab. There may be a concept or two which has not been introduced yet – in which case, I will leave a note about where the info is located so you can read it.

You may work on the labs (as soon as they are available). Students should aim to complete their labs before their lab due date and simply come to the lab session to demo their work. The lab sessions are for demos and help. Usually completing demos will be prioritized during labs - they take less time (and allow those who are complete to leave).

Due dates:

Labs are due in a week from when your lab started. For example: if your lab assignment is available Monday at 9am, then it is due in the lab on the following Monday. Late labs incur a 20% penalty per week. If a complete lab is a week late, it is worth 80%, if it is 2 weeks late, it is worth 60%, etc. You must complete all the labs to pass the course.

Marking:

I have found that it works best to do marking via visual inspection and demos during the lab period. If there are some improvements to be made – I'll let you know, and you have the choice to fix it and re-demo. If you're doing a lab demo that is late, it will be marked as it stands (without the chance to fix it for more marks).

Where to store your lab files:

In a previous semester Microsoft One was being used instead for student storage. Wherever you decide to store it is fine with me so long as you have access to it, and you're not able to lose it easily (as you might on a flash drive). You may want to back your files up from time to time just so you don't go through the ordeal of having to start over from scratch.

Microsoft Visual Studio Community

For the lab work in this course, we will be using Microsoft Visual Studio Community. This should already be installed on your lab computer. If you plan on working from home, this program is free. Downloading and installing it should be straightforward, but if you need assistance, let me know (or see LearnCpp section 0.6).

For PC: <https://visualstudio.microsoft.com/downloads/>

For Mac: [XCode](#)

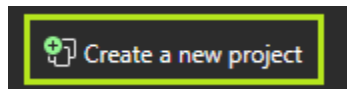
Upon installing, it may ask you if you which packages you want to install. You will want to specify (check) “Desktop development with C++”.

Login to your computer and Launch Visual Studio. You may find that upon starting, it requests you to login to a Microsoft Account. If you don’t have one already, a free Microsoft account can be created at: <https://account.microsoft.com/account?lang=en-hk>

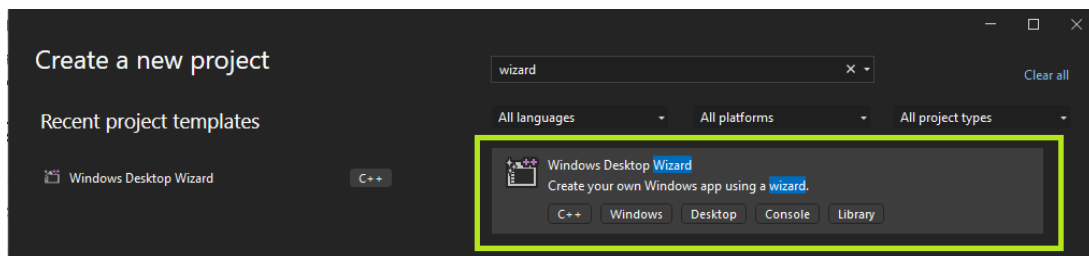
Creating a new project in Visual Studio (VS)

Steps for creating a new project were covered in “Chapter 0.7 – Compiling your first program” on LearnCpp.com.

Launch VS and click “Create a new project”.



Next select Windows Desktop Wizard for your template.

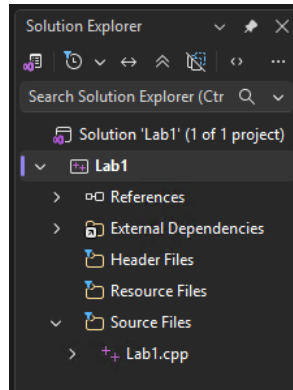


This step is important. If you don’t use the wizard you may end up with some odd errors (complaints about missing precompiled header PCH files).

A “Configure your new project” window will open. Give your new project the name: “Lab1” and click the “Create” button.

A “Windows Desktop Project” window will open. Keep the default options and click “OK”.

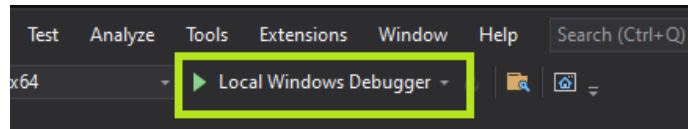
Because we didn’t opt for “empty project” on the last dialog, VS created a single source file: “Lab1.cpp” (visible in your Solution Explorer) and added it to your solution. [Note: If the Solution Explorer is not visible on the right side of the IDE, select View->Solution Explorer, and it should pop up \(see below\).](#)



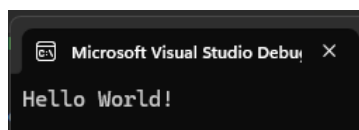
Have a look at the contents of Lab1.cpp. It should contain the following code:

```
4  #include <iostream>
5
6  int main()
7  {
8      std::cout << "Hello World!\n";
9  }
```

Save (CTRL-S), then build and run it by hitting F5 (or by clicking the green arrow just under the main menu)



If you see the following in your output window, it worked and you're ready to continue:



Objectives for this lab:

1. Write a basic .cpp file with a main function.
2. Use std::cout and std::cin for basic input and output.
3. Declare and use primitive variable types: int, double, char.
4. Use function return values
5. Pass parameters by value.
6. Use the srand() and rand() functions to get a random value.
7. Handle invalid text input.

Complete the following question(s). Test your work by ensuring that the output/input is the same as you see in the question examples.

Question 1

Create a new empty project called Lab1 in the exact same way that you created “Hello World C++”. Add a source file and call it lab1.cpp.

Step 1: - basic gameplay

Program Concept:

Write a game of “mystery number”. In this game, the player tries to guess a random number that the computer has picked (within a range). The computer tells the player “guess higher” or “guess lower” (depending the guess), and keeps track of the number of valid guesses it takes to guess the correct answer. Assume the minimum range is always 0. In this example, both 0 and 10 are valid guesses – anything outside this range is not.

Output from gameplay :

```
Microsoft Visual Studio Debug Console
*** Welcome to mystery number ***
Enter the maximum range: 10
Guess a number between 0 and 10: 5
guess lower...
Guess a number between 0 and 10: 3
guess lower...
Guess a number between 0 and 10: 2
Correct - it took you 3 guesses.
Do you wish to play again [y/n]: y

*** Welcome to mystery number ***
Enter the maximum range: 10
Guess a number between 0 and 10: 6
guess lower...
Guess a number between 0 and 10: 3
guess lower...
Guess a number between 0 and 10: 1
Correct - it took you 3 guesses.
Do you wish to play again [y/n]: n
Thanks for playing!
```

Do not use any global variables! (ie: don't use variables that are accessible to all functions without needing to pass it as a parameter). Functions should be passed the inputs they need, and then return a result (if necessary).

Note: As you write the functions below, test them to ensure they work as you expect. Do this before incorporating it into anything larger than itself. Temporarily call the function from `main()`, and then run the program to verify the result.

Write functionality for the following functions:

- `int promptUserForMaxRange()`
 - print "Enter the maximum range:"
 - read a number from the user
 - return the user specified max range from the keyboard
- `int pickRandomNumberInRange(int min, int max)`
 - return a random # between min & max (inclusive).
 - Eg: (1, 3) results in 1, 2 or 3.
 - ensure both min & max can be returned as potential values.
 - See more info on random numbers below (Generate a random number within a range).
- `int promptUserForGuess(int maxRange)`
 - print "Guess a number between 0 and `maxRange`"
 - return the user's guess from the keyboard
- `bool promptForPlayAgain()`
 - print "Do you wish to play again [y/n]:" to the console
 - read in the user's response
 - return true if the user enters 'y' or 'Y', and false if the user enters 'n' or 'N'
 - print "invalid input – try again" in all other cases and continues to iterate through the same question process until a valid response is made.
- `void playOneRound()`
 - this function should be responsible for co-ordinating a single round of gameplay
 - a round includes: the welcome message, asking the user for the range, selecting a random number, asking the user for the guesses, evaluating the guesses, tracking a count of guesses, prompting the user for a higher/lower guess, and printing "Correct, it took you # guesses!" once the correct answer is entered.
 - Make use of the functions listed above when possible.
- `int main()`

- use a loop to continuously play one round
- when the round is over, ask the user if they want to play again
 - If they do, stay in the loop
 - If they don't, print "Thanks for playing" and quit.

note: An **identifier** is the custom name that you as a programmer gives to a variable or a function. In C++, code gets compiled sequentially. In order for the compiler to recognize a function identifier, it must have encountered a declaration for the identifier prior to calling it. This can be accomplished in one of 2 ways:

1. Place the function definition prior to where it is called (do this for now).
2. Create a declaration for the function and place it prior to where it is called (we'll explore this more in the next lab)

Generate a random number within a range:

Note the following example for picking a random number:

```
int v1{ rand() % 100 };           // v1 in the range 0 to 99
int v2{ 1 + (rand() % 100) };     // v2 in the range 1 to 100
int v3{ 1985 + (rand() % 30) };   // v3 in the range 1985-2014
```

The `rand()` function exists in a header file called `<cstdlib>`. It automatically gets included when we include `<iostream>`, but it's good practice to include it anyway (in case we ever replaced the `cout` statements with a GUI and didn't need `iostream` anymore).

The `rand()` function will return a pseudo-random number between 0 and `RAND_MAX` (a constant in `cstdlib` that is typically set to 32767). To get a random number within a range, we can use the mod (%) operator to return the remainder of integer division.

```
9 / 4 == 2  // the result of the division
9 % 4 == 1  // the remainder
```

If we want a random number in the range 0...3, all we have to do is take a random int and mod it by the number at the end of the range + 1 (4 in this case).

`rand() % 4 == a random number in the range [0..3]`

For this game we will only be picking numbers between 0 and N. However, `pickRandomNumberInRange()` should be flexible enough to work for any given range. Ensure it works when the first parameter is something other than 0. Find a way to do this and then prove it works by writing a temporary for loop in your main program and printing

the result of `pickRandomNumberInRange(5, 7)` twenty times to ensure all values (5,6,7) are showing up randomly without any off-by-one errors. About 60% of past students tend to miss this. Get in the habit of proving to yourself that your individual functions work and you'll avoid having to track down problems in larger programs. Bugs are easier to catch when the code is limited in size.

Looping and if-else control structures:

Loops and if else control structure in C++ are identical to the Java equivalents.

Build and run shortcuts:

Once you have a workable solution in place, build and run (shortcut: F5).

Note: Hitting F5 or clicking the play button beside "Local Window Debugger" will run your program in debug mode. The console may execute and then disappear without giving you a chance to see the output! This can be adjusted by deselecting the checkbox in "Tools | Options | Debugging | Automatically close the console when debugging stops".

Run it a second time, and then a third time. Did you notice anything odd? The pseudo-random numbers that are generated are the same for every run!

Seeding the randomizer:

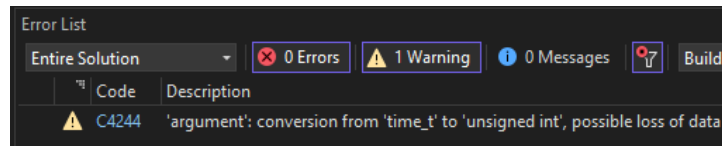
To get different random numbers every time we execute the application, we need to "seed our randomizer" (ie: initialize it) with a number that it uses to build its own random numbers. **This only needs to be done once at the start of `main()`.** Seeding the randomizer multiple times will cause problems with the randomness of the numbers generated. Once the randomizer has been seeded, each new call to `rand()` will give a new quasi-random number.

We can seed the randomizer by calling the `srand(n)` function, where `n` is some integer that changes every time we run the program. An easy to provide a unique `n` is by using the `time(0)` function which resides in the `<time.h>` header (you will need to `#include` this). Calling `time(0)` or `time(NULL)` will return the number of seconds that have passed since 00:00 hours on Jan 1, 1970. This will be different for every run of the program.

Try adding the following at the top of your `main()`:

```
srand(time(0));
```

This should compile. It **may** give a warning because the type of the value returned by `time(0)` is called `time_t` and not an integer as you might expect.



This variable of type `time_t` is getting converted implicitly (automatically) to an unsigned int since that is what `srand()` is expecting as a parameter.

To get rid of the warning, we can tell the compiler explicitly to convert the `time_t` type that is returned from `time(0)` to an unsigned int as follows:

```
srand( static_cast<unsigned int>(time(0)) );
```

Trust that this little bit of ‘magic’ should work for now – and we will explain casting in a later class.

Note that the same conversion that was happening before is happening here, but the compiler isn’t giving us a warning anymore because we were explicit about the conversion (rather than it just happening implicitly). We’re essentially telling the compiler we know about the conversion and that we intend for it to happen.

Step 2 – DRY principle & function reuse:

Notice that `promptUserForMaxRange()` and `promptUserForGuess()` have similar code in them (with the exception of the prompt given to the user). In the next step we’re going to handle incorrect input when reading in an integer. Currently, you would need to add error checking in both functions. Remember the DRY (don’t repeat yourself) principle? How could you reduce the duplication here and make it so that error checking would only need to happen in one place? Think about it for a minute and see if you can come up with something. Give it a try to see if it works before checking the solution on the next page.

You can eliminate the duplication by moving the logic that reads an int from the user into a new function. Write a function called `getIntFromUser()` (which simply reads in and returns an int that the user enters). Have the 2 existing prompting functions both call it. When we do error checking on the input (step 3) then we only need to do it in one place instead of multiple locations.

Step 3 – handling invalid input:

What happens if input is incorrect? Try entering a character instead of an integer to see what happens?

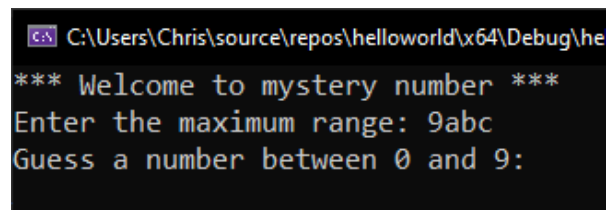
Read through all of [9.5 `std::cin` and handling invalid input](#) on LearnCpp.com. It describes 4 types of invalid text input that could occur for `getIntFromUser()`. It will show you how to handle errors 2, 3, and 4.

- **Error case 1:** Input extraction succeeds, but input is meaningless to the program (a guess is entered that is out of range – like: -1)
- **Error case 2:** Input extraction succeeds but the user enters additional input (eg: entering '9abc')
- **Error case 3:** Input extraction fails (eg: trying to enter 'abc' as numeric input)
- **Error case 4:** Input extraction succeeds, but the user overflows a numeric value (eg: entering 999999999999)

Error case 1 deals with a situation where `getIntFromUser()` worked just fine, but didn't work within the context of the game because the number is out of range (according to the rules of the game). Let's leave this for now and come back to it in a moment.

Error cases 2, 3, and 4 all deal with situations where the `getIntFromUser()` function could fail. Modify `getIntFromUser()` to deal with these three cases.

If error case 2 happens, the program should ignore the rest of the line and return the integer that was at the front.



```
C:\Users\Chris\source\repos\helloworld\x64\Debug\he
*** Welcome to mystery number ***
Enter the maximum range: 9abc
Guess a number between 0 and 9:
```

Error case 2 handled correctly

If error case 3 or 4 happens, the program should output `""Invalid input. Input must be a valid int. Please try again."` to the console. The program should then continue to wait for valid input. You should be able to do this using a `while( true ) {...}` loop inside your `getIntFromUser()` function. You can use a return statement inside the loop once you have a valid int.

I realize (from previous classes) that the idea of a `while(true) {...}` loop with an exit condition that returns from the function (or breaks out of the loop) may go against some things you've learned in first year. In game programming it is common to have loops that you break out of when a certain condition is met from inside the loop. My personal rule for how to handle things like this is – "Write it in a way that makes it both extensible and easy to understand". As a developer you will likely spend 20% of your time writing code, and 80% of your time reading and trying to understand code (someone else's or your own) so that you don't break it with your changes. The easier the code is to understand (and potentially extend), the less chance of introducing bugs.

```
C:\Users\Chris\source\repos\helloworld\x64\Debug\helloworld.exe
*** Welcome to mystery number ***
Enter the maximum range: abc
Invalid input. Input must be a valid int. Please try again.
abc
Invalid input. Input must be a valid int. Please try again.
9
Guess a number between 0 and 9:
```

Error case 3 handled correctly

```
C:\Users\Chris\source\repos\helloworld\x64\Debug\helloworld.exe
*** Welcome to mystery number ***
Enter the maximum range: 999999999999
Invalid input. Input must be a valid int. Please try again.
8
Guess a number between 0 and 8:
```

Error case 4 handled correctly

`GetIntFromUser()` should now be working correctly. Did you notice that if you hadn't migrated duplicate code into a common function that you would need to do error checking for reading integers in multiple places? Eliminating the redundancy/repetition in the code has saved you writing this functionality multiple times. You've now also got a function that can be used in the future whenever you want to prompt for an integer. It's got error checking built in and has already been tested. Nice!

Let's now deal with **error case 1**. There are 3 situations that need attention:

1. In `promptUserForMaxRange()`, the value entered should be > 0 , but $\leq$ the maximum value an integer can hold (you can use an existing preprocessor definition of `INT_MAX` for this purpose).
2. In `promptUserForGuess(maxRange)` the value entered should be between $[0 \dots \text{maxRange}]$ (inclusive).
3. In `promptForPlayAgain()` the value entered should be `y`, `Y`, `n`, or `N`.

Let's handle the first two situations by writing another function called `getIntFromUserInRange(min, max)`. Inside this function we can call `getIntFromUser()` so we don't have to re-write the error checking. However, we will also need to add logic to ensure that the int entered is within $[\text{min} \dots \text{max}]$ range. If it is out of range, output the message: **"Invalid input. Range is [min to max]. Please try again."** You can use another `while(true) {...}` loop in this function to continue attempting to read an int that is in range. Simply return when an acceptable int is entered. Update the existing calls to `getIntFromUser()` to use `getIntFromUserInRange()` for situations 1 & 2 listed above and you should get output that looks similar to the following:

```
*** Welcome to mystery number ***
Enter the maximum range: 0
Invalid input. Range is [1 to 2147483647]. Please try again.
999999999999999
Invalid input. Input must be a valid int. Please try again.
5
Guess a number between 0 and 5: -1
Invalid input. Range is [0 to 5]. Please try again.
6
Invalid input. Range is [0 to 5]. Please try again.
3
guess higher...
Guess a number between 0 and 5: 5
Correct - total guesses: 2
Do you wish to play again [y/n]: n
Thanks for playing!
```

Let's now turn our attention to situation 3. In `promptForPlayAgain()`, make it so that if the user enters anything other than the characters: `y`, `Y`, `n`, `N`, the player gets prompted with **"Invalid input. Try again."**, and continues to prompt the user until it gets a valid response. You can use another `while(true) {...}` loop here if you wish. It should perform as shown below:

```

*** Welcome to mystery number ***
Enter the maximum range: 1
Guess a number between 0 and 1: 1
Correct - total guesses: 1
Do you wish to play again [y/n]: 1
Invalid input. Try again.
Do you wish to play again [y/n]: A
Invalid input. Try again.
Do you wish to play again [y/n]: y

*** Welcome to mystery number ***
Enter the maximum range:

```

You're almost done. Go through your code and add a comment at the top of every function stating exactly what the function does, what its expected inputs and outputs are, etc. This is a best practice that I do to make things easier for both myself and others later on.

Before doing a demo

- Ensure your program responds correctly to all the invalid input cases shown below.
- Ensure the total guesses count works correctly (it only counts guesses for valid input)

```

*** Welcome to mystery number ***
Enter the maximum range: 0
Invalid input. Range is [1 to 2147483647]. Please try again.
999999999999999999
Invalid input. Input must be a valid int. Please try again.
abc
Invalid input. Input must be a valid int. Please try again.
3abc
Guess a number between 0 and 3: -1
Invalid input. Range is [0 to 3]. Please try again.
4
Invalid input. Range is [0 to 3]. Please try again.
abc
Invalid input. Input must be a valid int. Please try again.
2abc
guess higher...
Guess a number between 0 and 3: 3
Correct - total guesses: 2
Do you wish to play again [y/n]: 9
Invalid input. Try again.
Do you wish to play again [y/n]: abc
Invalid input. Try again.
Do you wish to play again [y/n]: Y

*** Welcome to mystery number ***
Enter the maximum range:

```

Code Review

When you are finished this assignment, find a friend who is also finished. Using a LabCodeReview checklist (available in the lab), conduct a code review on each other's code **before** giving a demo of your assignment for marks.

Marking [out of 10 points]

- Step 1: [4 pts]
 - Basic gameplay implemented without bugs,
 - Ranges work properly when correct data is entered.
 - `pickRandomNumberInRange(5,7)` works as expected (even though it the range doesn't start at zero).
- Step 2: [1 pts]
- Step 3: [4 pts]
 - one point per error case handled
- [1pt] A comment at the top of every function describing what it does.

If you are done within lab time, request a demo and I can sign off on it. If not, I can check it at the beginning of the next lab, or you can schedule a time to demo it.

Labs are due a week from the lab block in which they are assigned (at midnight). Eg: if you are working on a lab that was "released" in week 2 and your lab is on a Thursday then it is due in the lab on the following Thursday. Late labs will incur a penalty of 20% per week.